

Hochschule für Technik
und Wirtschaft Berlin

University of Applied Sciences

IoT Environmental Monitoring with Telegram

White Paper

Master Professional IT Business & Digitalization

First Supervisor: Prof. Dr. Alexander Huhn

Submitted by: Rohan Panda

Matriculation Number: s0595920

Submission Date: 13.01.2025

INDEX

CONCLUSION 21
DATA PROCESSING 7
DATA PROCESSING
Basic Alert on crossing threshold value ..... *See*
Configuration of MQTT broker in Node Red 7
Data Flow through different Nodes in Node-RED 8
Displaying of average values in a pop-up and within a TEXT node on the Dashboard 14
Feature Implementation 10
Hot and Cold Alert for crossing threshold value for continuous increase or decrease 11
Setting User Defined Threshold from Telegram 13
Telegram Chatbot with Menu Options 12
Text-to-Audio functionality on the dashboard 14
DATA STORAGE 16
DATA STORAGE
Data Storage Flow 17
GITLAB REPOSITORY 21
HARDWARE COMPONENT 4
HARDWARE COMPONENT
DHT22 Sensor 4
ESP8266 4
Raspberry Pi 4
INTRODUCTION 3
INTRODUCTION
PROJECT PURPOSE 3
PROJECT SCOPE 3
MICROPYTHON CODE 6
REFERENCES 21
UI AND VISUALIZATION 18
UI AND VISUALIZATION
Data Visualization in Influx DB: 19
Data Visualization in Node-RED Dashboard 18

1. INTRODUCTION

This project aims to integrate IoT components, including a DHT22 sensor, a Raspberry Pi, and an ESP8266 microcontroller, with a Telegram chatbot to enable real-time measurement and reporting of temperature and humidity. The system utilizes sensors to collect environmental data in real time and delivers updates through the Telegram chatbot, ensuring convenient access to information and customizable alerts.

1.1. PROJECT PURPOSE

This document provides a comprehensive overview of the project's use case and explains how various components—including microcontrollers, sensors, data transmission, data processing, data visualization, and alarm systems—were integrated to achieve its objectives. It also serves as a reference and foundation for future improvements and enhancements to the system.

1.2. PROJECT SCOPE

The scope of this project is to design and implement a user-friendly and efficient Telegram chatbot for monitoring temperature and humidity in controlled environments. The chatbot will serve as a convenient and accessible tool for real-time monitoring and for modifying threshold values for temperature and humidity as needed.

2. HARDWARE COMPONENT

For this project, we have used a Raspberry Pi (Fig. 2.1.), DHT22 Module and ESP8266 Microcontroller module (Fig. 2.2.).

2.1. Raspberry Pi

The Raspberry Pi is a debit card-sized, low-cost, and low-powered computer that connects to a computer desktop or TV and uses a standard mouse and keyboard. In this project, it is used for data collection, processing, storage, visualization, alerts, and user communication.

2.2. DHT22 Sensor

The DHT22 is a simple temperature and humidity sensor with a digital display. It measures the ambient air with a capacitive humidity sensor and a thermistor and outputs a digital signal on the data pin, requiring no analog input pins.

No	Pin Name	Description
1	VCC	It has a power supply of 3.5V to 5.5V.
2	Data	The temperature and humidity output signals via serial data.
3	Ground	The module usually connects to the ground of a circuit.

Table 2.1. Pin Configuration of DHT22 sensor

2.3. ESP8266

The ESP8266 is an affordable WiFi-enabled microcontroller developed by Espressif Systems, a technology company based in Shanghai, China. In this project, the 3.3V, D2, and GND pins of the ESP8266 are connected to the VCC, Data, and Ground pins of the DHT22 sensor, respectively.

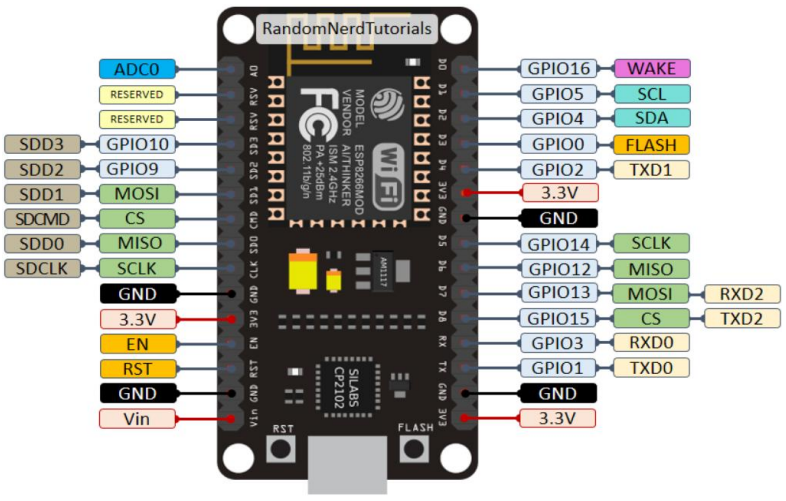


Fig 2.1. Pin diagram of ESP8266 Microcontroller

Pictures of Raspberry Pi (Fig. 2.1.) and the DHT22 sensor connected to the ESP8266 Microcontroller module (fig. 2.2.).



Fig. 2.2. Raspberry Pi



Fig. 2.3. ESP 8266 MC and DHT22 Sensor

No	DHT22 PIN	ESP8266 MC PIN
1	VCC	3 .3 V
2	Data	D2 (Pin 2)
3	Ground	GND

Table 2.3. Pin mapping of ESP8266 Microcontroller and DHT22 sensor

3. MICROPYTHON CODE

The MicroPython code is written using Thonny and saved in the boot.py file of the ESP8266 microcontroller (Fig. 3.1). The do_connect() function ensures the microcontroller is connected to Wi-Fi, and it must be called before performing any other operations.

The following code snippet is used to measure data from the DHT22 sensor:

```
# Specify the ESP8266 pin connected to the DHT22 output
d = dht.DHT22(machine.Pin(2))
# Prompt the sensor to measure the data
d.measure()
# Retrieve temperature and humidity readings from the sensor
d.temperature()
d.humidity()
```

For MQTT communication, the following code initializes the MQTT client and publishes sensor data:

```
# Initialize the MQTT client and connect to the server
client = MQTTClient("ProIT_Rohanclient", "broker.f4.htw-berlin.de")
# Publish temperature and humidity data to specific topics
client.publish("HTW_IoTM2425/Rohan/temp", b"%s" % d.temperature())
client.publish("HTW_IoTM2425/Rohan/humi", b"%s" % d.humidity())
```

boot.py

```
# This file is executed on every boot (including wake-boot from deepsleep)
import network
import dht
import machine
from umqttsimple import MQTTClient
import time

def do_connect():
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)
    if not wlan.isconnected():
        print('connecting to network...')
        wlan.connect('HTW_H106_Rechnernetze', 'rnFIW625')
        while not wlan.isconnected():
            pass
    print('network config:', wlan.ipconfig('addr4'))

do_connect()

d = dht.DHT22(machine.Pin(2))
d.measure()
d.temperature()
d.humidity()

client = MQTTClient("ProIT_Rohanclient", "broker.f4.htw-berlin.de")

client.connect()
while True:
    time.sleep(20)
    print("Measuring Temperature and Humidity")
    d.measure()
    client.publish("HTW_IoTM2425/Rohan/temp", b"%s" % d.temperature())
    client.publish("HTW_IoTM2425/Rohan/humi", b"%s" % d.humidity())
```

Fig 3.1. Boot.py file which was programmed in ESP8266 MC

4. DATA PROCESSING

Data processing in IoT involves the techniques and methods used to collect, transform, analyze, and leverage the large volumes of data produced by IoT devices and sensors. For this project, we are collecting data from sensors and transformation is done in Raspberry Pi.

4.1. Configuration of MQTT broker in Node Red

The MQTT IN node of Node-RED is used to subscribe to topics published to the server `**broker.f4.htw-berlin.de**`. The ESP8266 microcontroller operates as a standalone device, running a MicroPython `boot.py` file. It connects to the `broker.f4.htw-berlin.de` server, hosted on campus, and publishes temperature and humidity data collected from the DHT22 sensor. The data is transmitted to a Raspberry Pi running Node-RED.

The ESP8266 measures the latest temperature and humidity values every 20 seconds and publishes them to the respective topics: `**HTW_IoTM2425/+/temp**` and `**HTW_IoTM2425/+/humi**` using the MQTT protocol. Additionally, temperature and humidity data are also published to specific topics, such as `**HTW_IoTM2425/Rohan/temp**` and `**HTW_IoTM2425/Rohan/humi**`.

The below two figures (Fig. 4.1. and Fig. 4.2.) show the configuration of subscribing to multiple topics and a single topic, respectively.

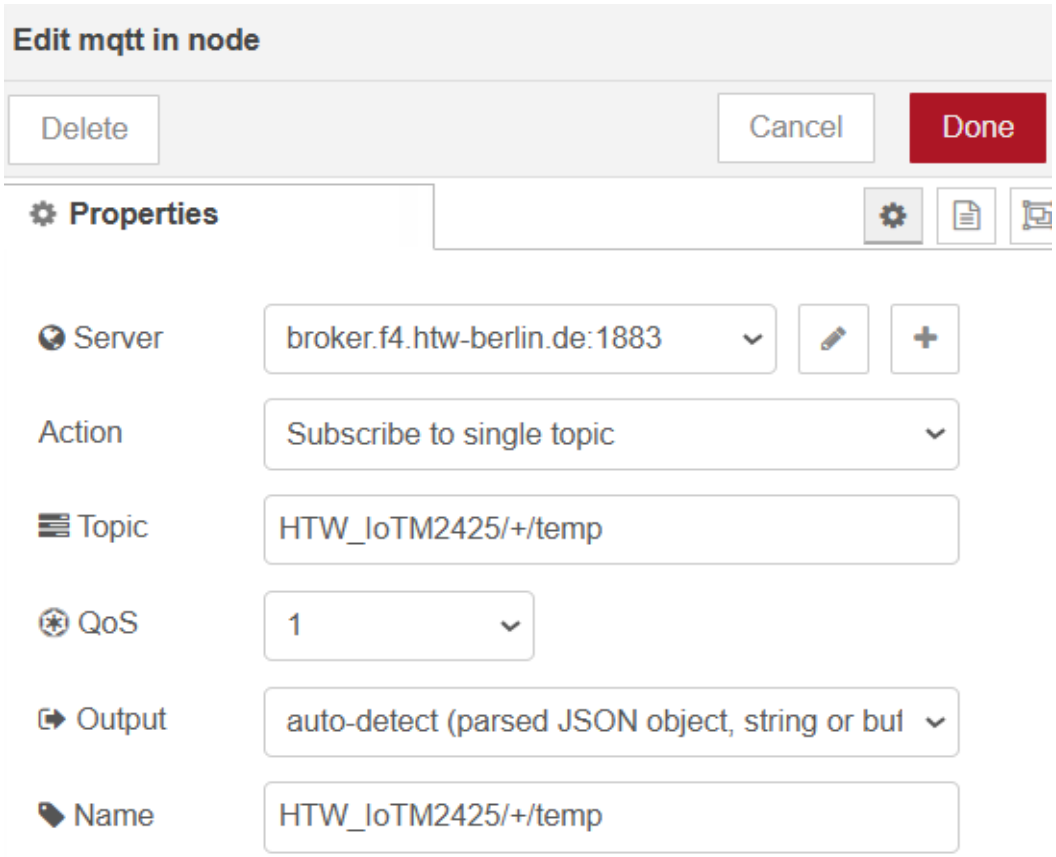


Fig 4.1. Configuration of MQTT Broker for subscribing to Multiple topic

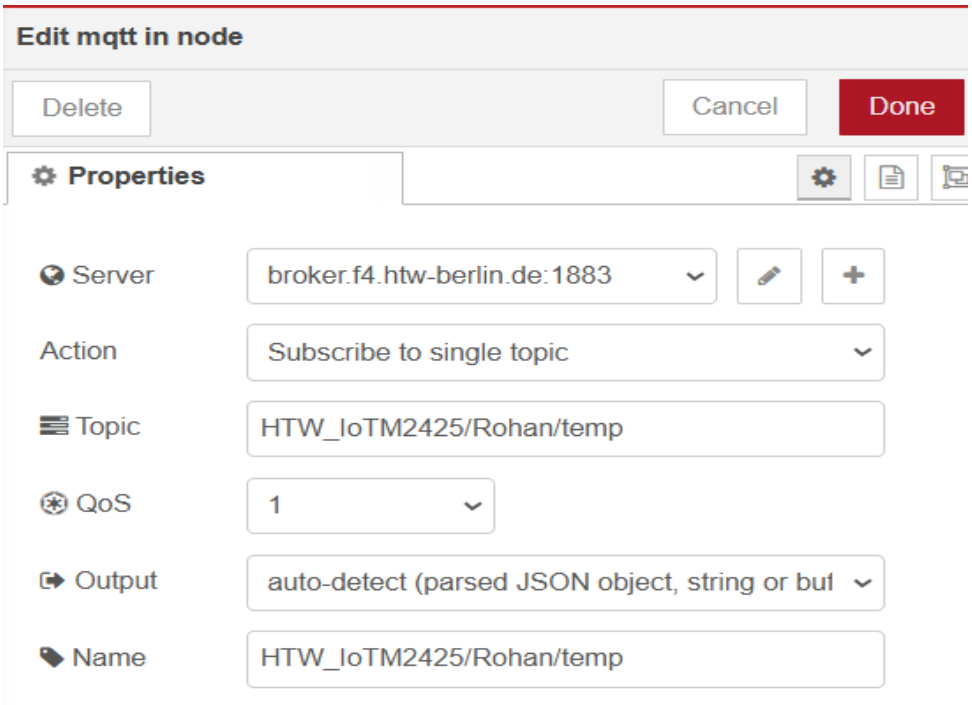


Fig 4.2. Configuration of MQTT Broker for subscribing to Single topic

4.2. Data Flow through different Nodes in Node-RED

Figs. 4.4. and 4.5. illustrate how data is subscribed to using either a wildcard or specific topics via the MQTT IN node. The subscribed data is then directed to Charts and Gauges for visualization in the Node-RED Dashboard. In another flow, the data undergoes conditional checks at the SWITCH node before being processed for string operations in the STRING node. Subsequently, the data is formatted into a proper JSON structure using the CHANGE node and forwarded to the Telegram Bot through the TELEGRAM SENDER node, which sends messages to the user.

We can achieve the same function which was used in SWITCH, STRING and CHANGE node by using FUNCTION node (Fig. 4.3.).

```
var utemp = flow.get('userTemp') || 0;
var sensorTemp = msg.payload;

if (utemp != 0 ){
    if (sensorTemp > utemp) {
        msg.payload = {
            chatId: 700782238,
            type: 'message',
            content: `Alert !!! Temperature is more than threshold. It is ${sensorTemp} now. Please check it !!`
        };
        return msg;
    }
} else {
    if (sensorTemp > 20) {
        msg.payload = {
            chatId: 700782238,
            type: 'message',
            content: `Alert !!! Temperature is more than threshold. It is ${sensorTemp} now. Please check it !!`
        };
        return msg;
    } else {
        return null;
    }
}
```

Fig. 4.3. Code to replace the use of SWITCH, STRING and CHANGE node

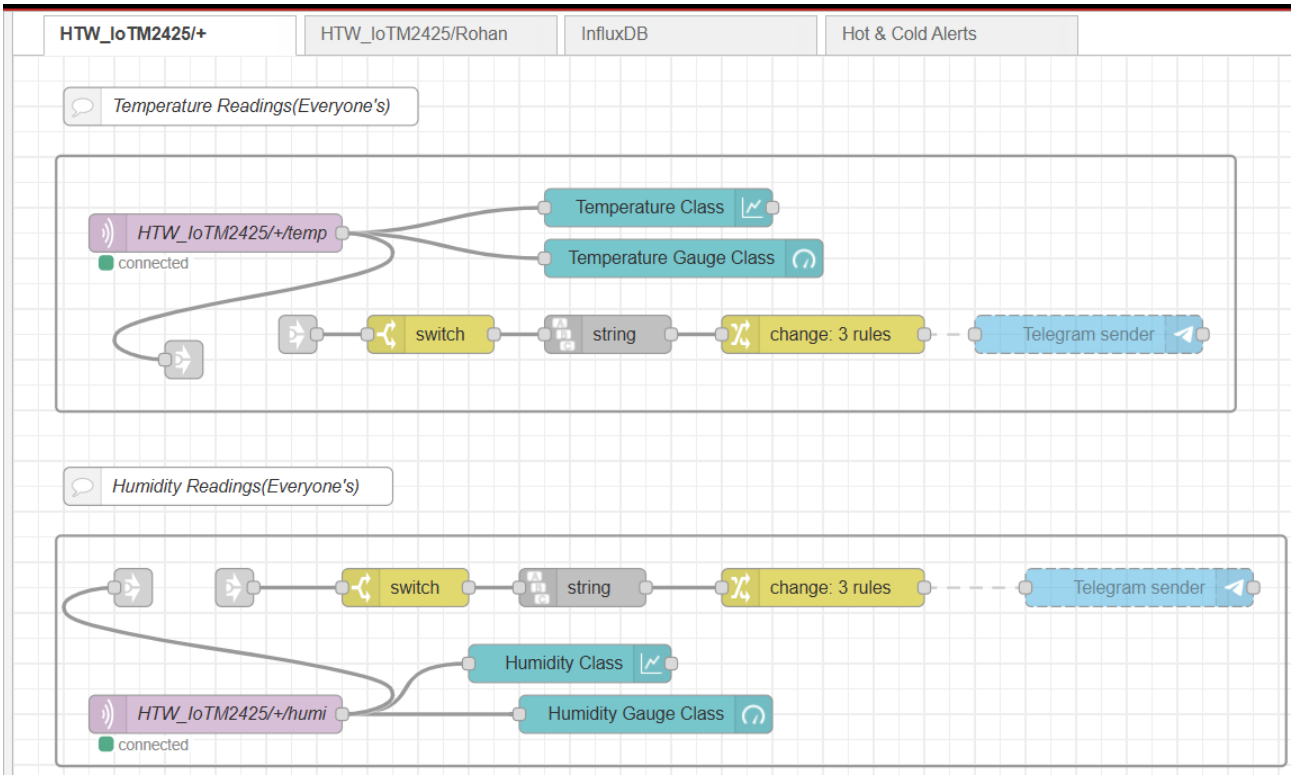


Fig 4.4. Shows the connection of nodes for publishing of everyone's topic to Node-Red Dashboard and Telegram Alerts

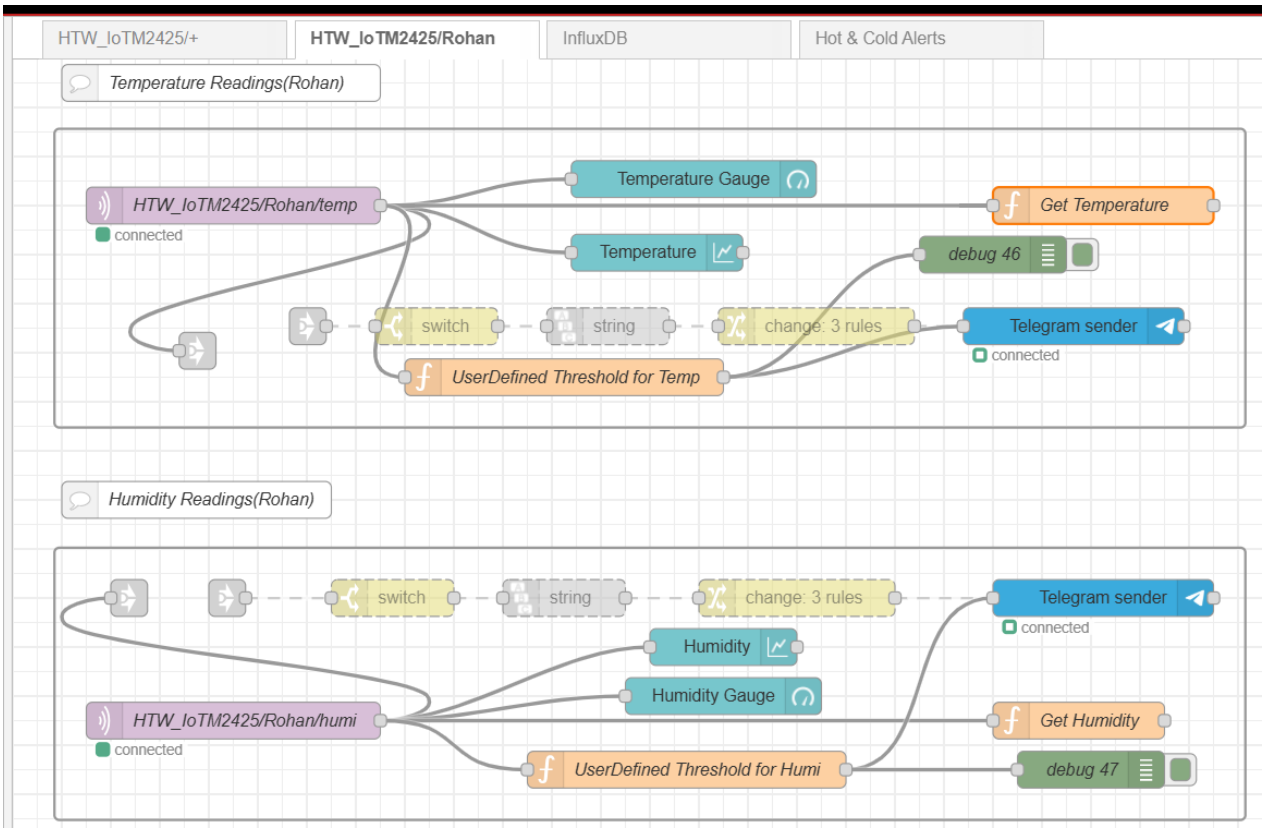


Fig 4.5. Connection of nodes for publishing of Rohan topic to Node-Red Dashboard and Telegram Alerts

4.3. Feature Implementation:

This IoT project offers several additional features, including alerts for continuous increases or decreases in temperature and humidity, the ability to set user-defined threshold values for these parameters via Telegram, an interactive chatbot with menu options, text-to-speech functionality on the dashboard, and the option to display average values in a pop-up and within a TEXT node on the dashboard.

4.3.1. Basic Alert on crossing threshold value

The basic alert (Fig. 4.6.) is triggered when the temperature or humidity value exceeds a specified threshold. An alert message is then sent to the user via the Telegram chatbot. Refer to Figure 4.5 for the corresponding Node-RED flow.

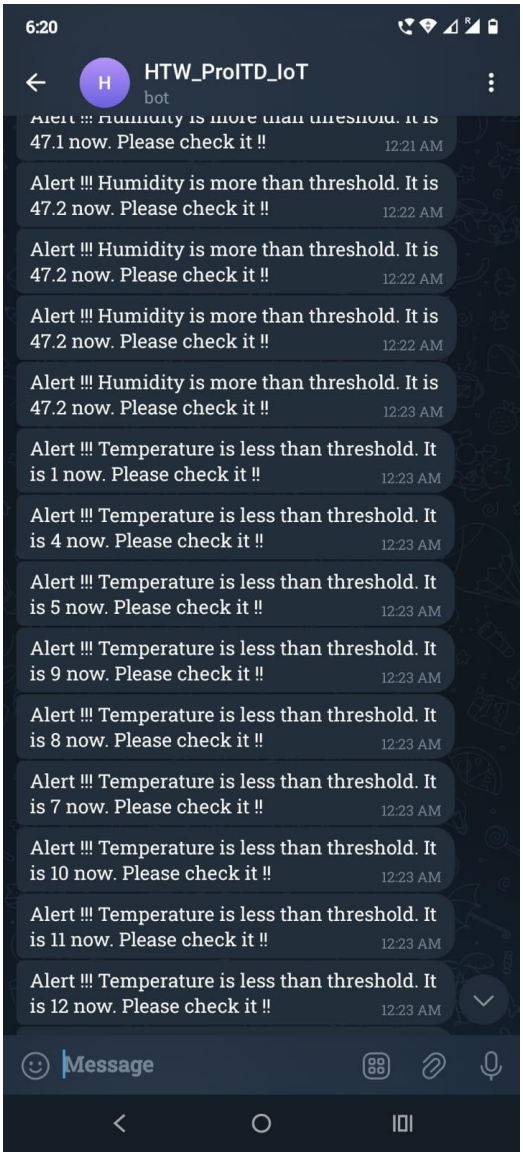


Fig 4.6. Basic Alert

4.3.2. Hot and Cold Alert for crossing threshold value for continuous increase or decrease

Hot and Cold alerts are triggered only when a specified threshold value is breached and remains at or continues increasing/decreasing for five or more consecutive sensor measurements (Check Highlighted parts of Fig. 4.7.).

- Hot Alert: This is triggered when the temperature exceeds 24°C and stays at or above this value for five or more measurements. An alert is then sent to the user via Telegram, including the total time of occurrence.

- Cold Alert: This is triggered when the temperature falls below 11°C and remains at or below this value for five or more measurements. A Telegram alert is sent to the user with the corresponding time details.

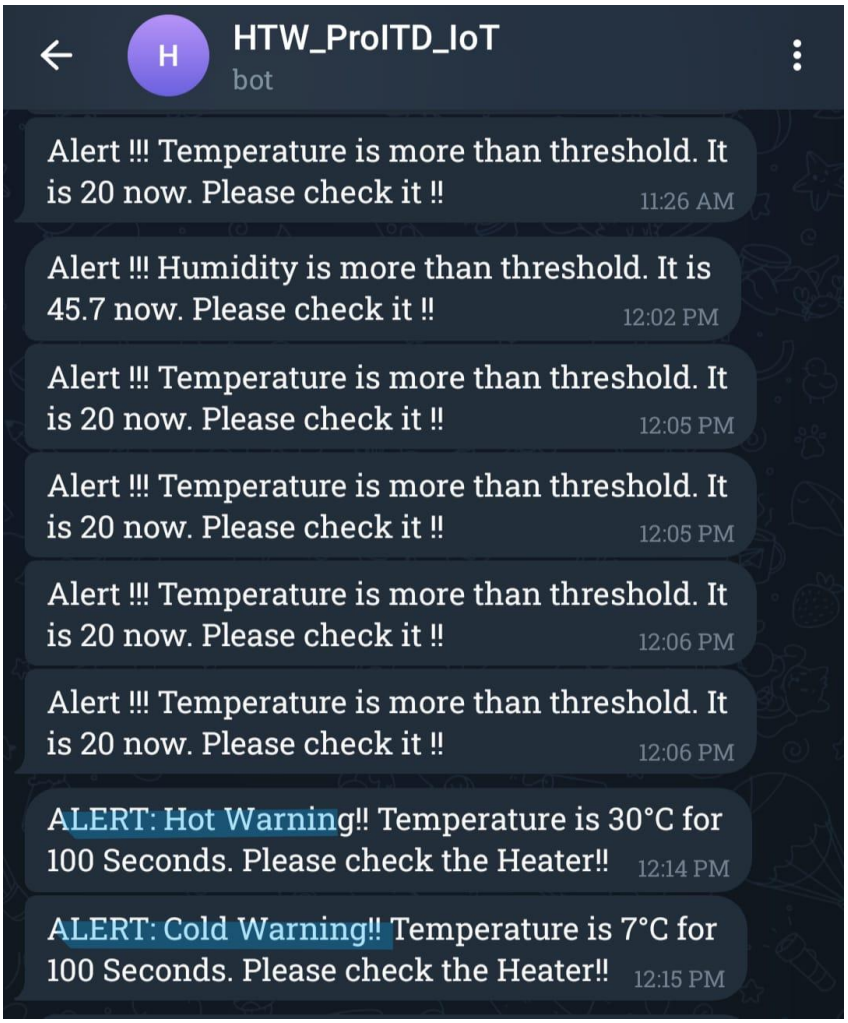


Fig 4.7. Flow for continuous Increase/Decrease

Fig. 4.8 illustrates the Node-RED flow used to implement the Hot and Cold Alerts. The LINK IN node receives temperature values from the MQTT IN node. In the first FUNCTION node, the current temperature is stored in a flow variable, which is then passed to the second FUNCTION node. The second node evaluates the specified conditions for Hot and Cold Alerts and sends an

alert to Telegram if the conditions are met.

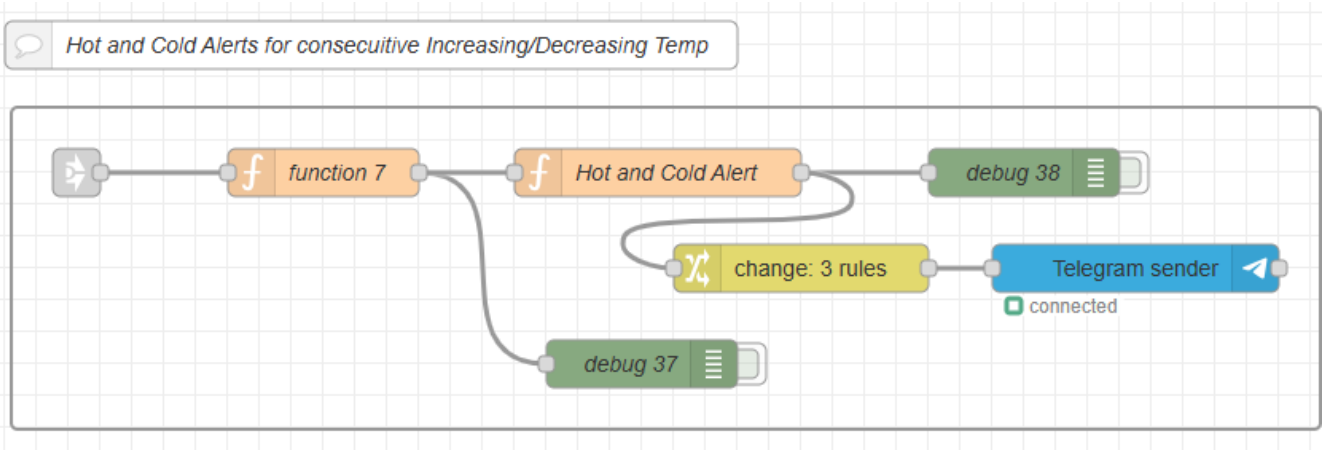


Fig 4.8. Flow for continuous Increase/Decrease

4.3.3. Telegram Chatbot with Menu Options

Fig. 4.9 depicts the Node-RED flow for implementing menu-like customization in the Telegram chatbot, while Figs. 4.10 and 4.11 showcase the display of menu options and their usage, respectively.

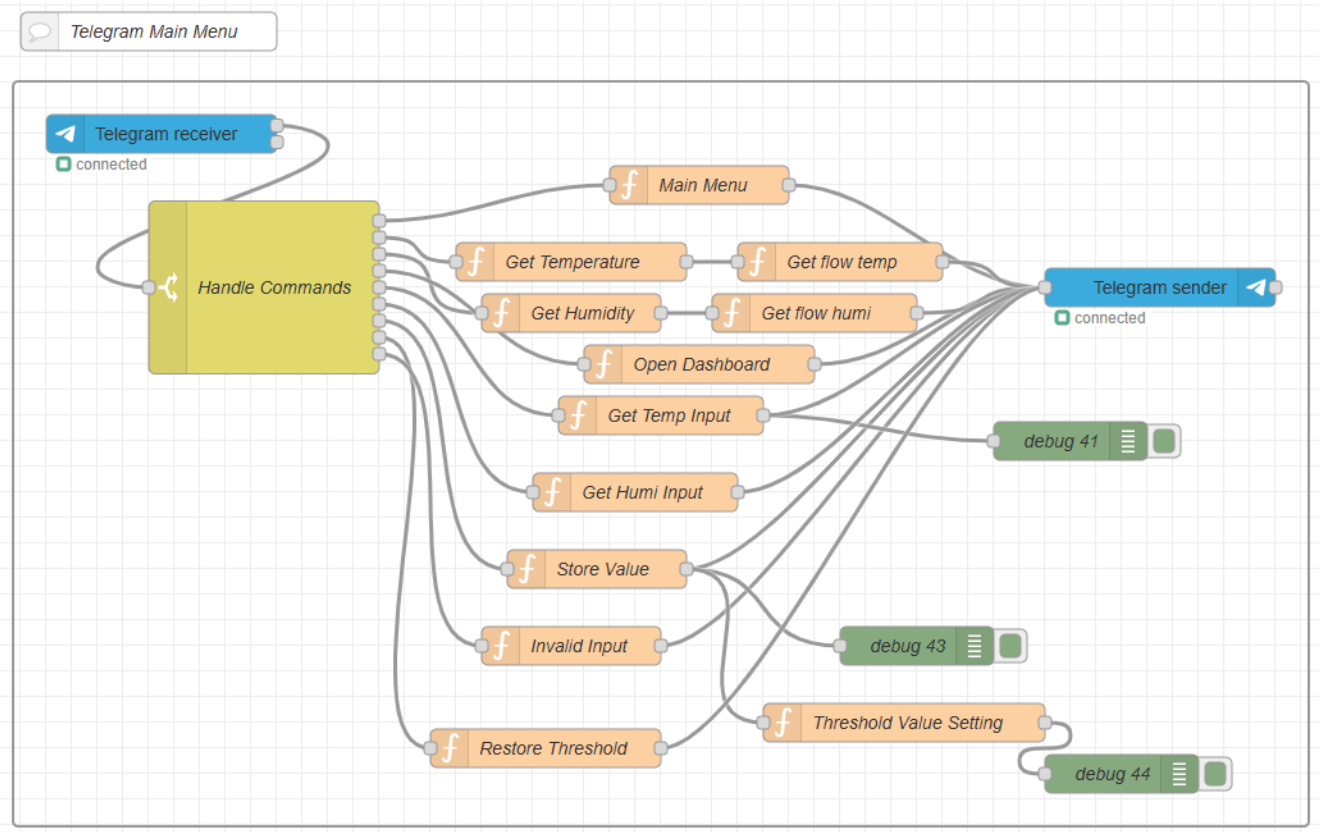


Fig. 4.9. Telegram Chatbot's Menu Flow in Node-RED

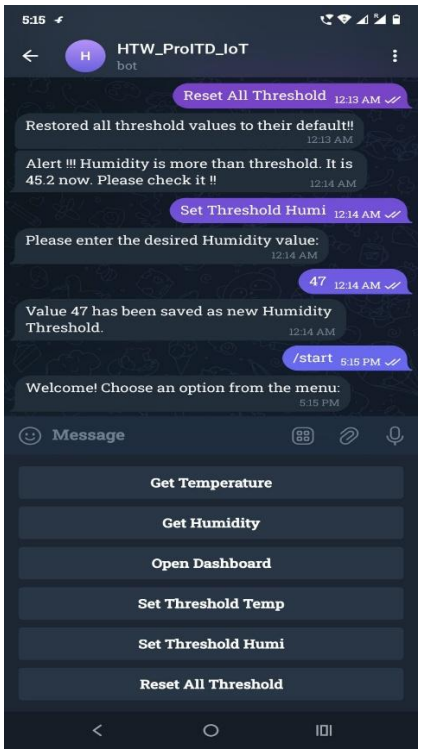


Fig 4.10. Telegram Chatbot's Menu

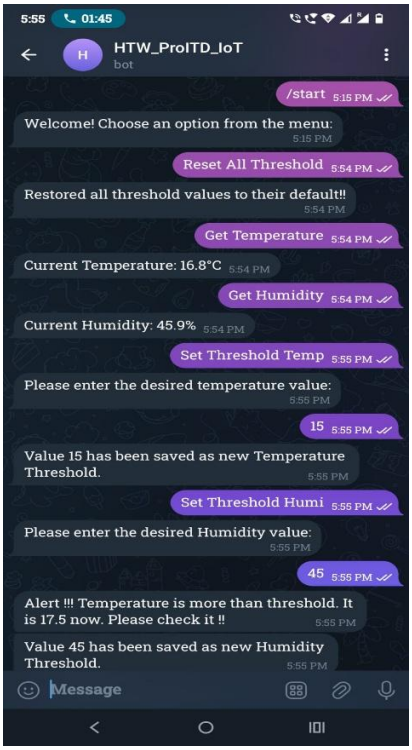


Fig 4.11. Telegram Chatbot's Menu being used

4.3.4. Setting User Defined Threshold from Telegram

For this, refer to Figure 4.9, where, after the user selects an option to set a threshold value for temperature, humidity, or both, the default thresholds for these parameters are overwritten. This is accomplished using global and flow variables.

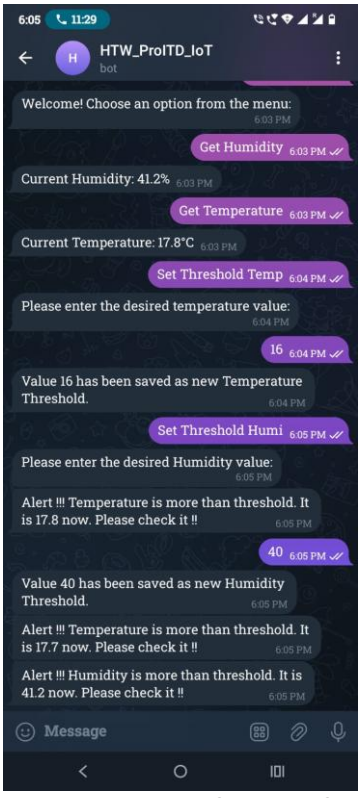


Fig 4.12. Setting of User Defined Threshold

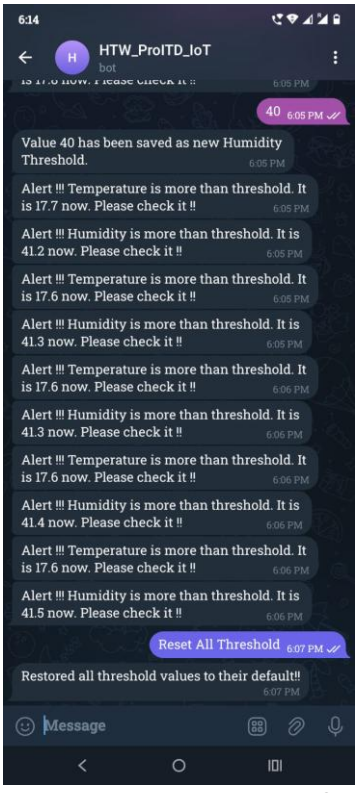


Fig 4.13. Resetting User Defined Threshold Values

4.3.5. Text-to-Audio functionality on the dashboard

Fig. 4.14 illustrates the Node-RED flow for text-to-audio conversion. A BUTTON node is used to display a button on the dashboard, which, when clicked, triggers a FUNCTION node. The FUNCTION node retrieves the current and average values for temperature and humidity. This text payload is then converted to audio using the AUDIO OUT node.

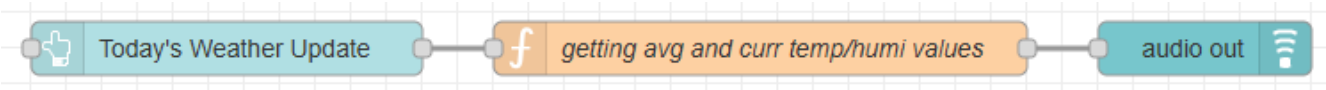


Fig 4.14. Flow to get the average and give audio response

4.3.6. Displaying of average values in a pop-up and within a TEXT node on the Dashboard

Figure 4.16 illustrates how average values are calculated by retrieving all temperature and humidity data stored in InfluxDB. This is achieved by executing a flush query to fetch the data, which is then passed to a FUNCTION node where string operations are performed. The average values obtained from InfluxDB are displayed as text using the TEXT node.

Figure 4.15 shows the Node-RED flow where the BUTTON node, when clicked, triggers the FUNCTION node to retrieve the average values from Figure 4.17 and display them in the NOTIFICATION node.

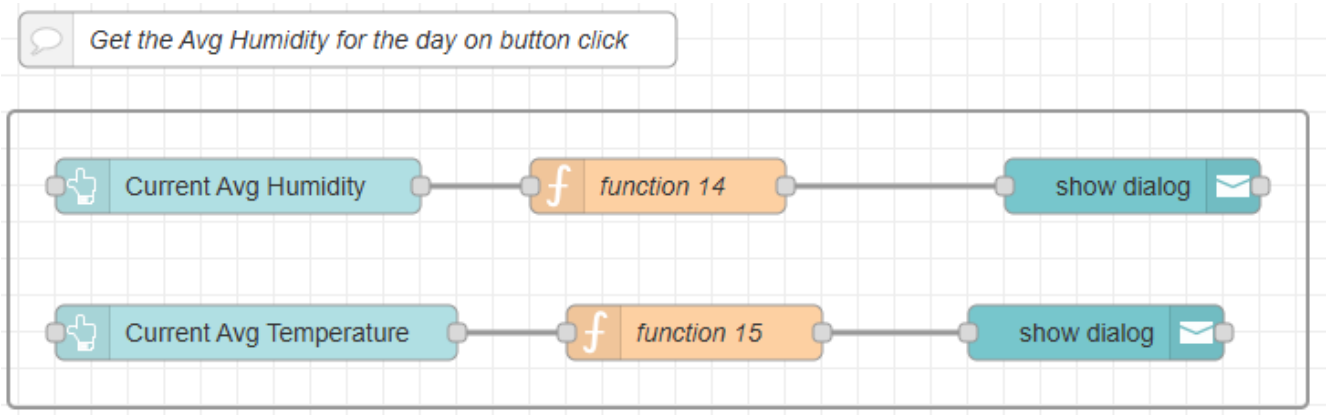


Fig 4.15. Flow to get the average and give audio response

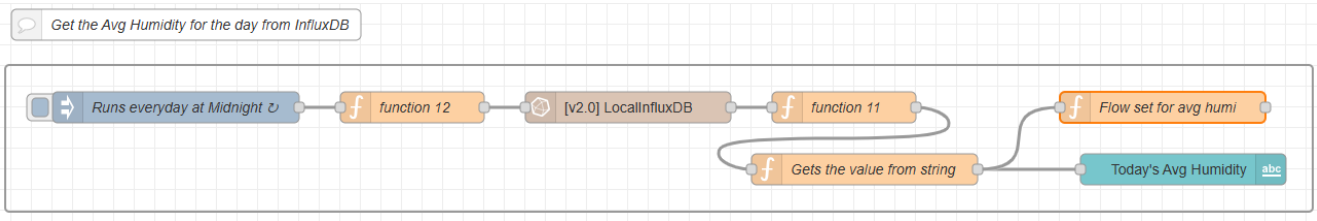


Fig 4.16. Flow to get the average and give audio response

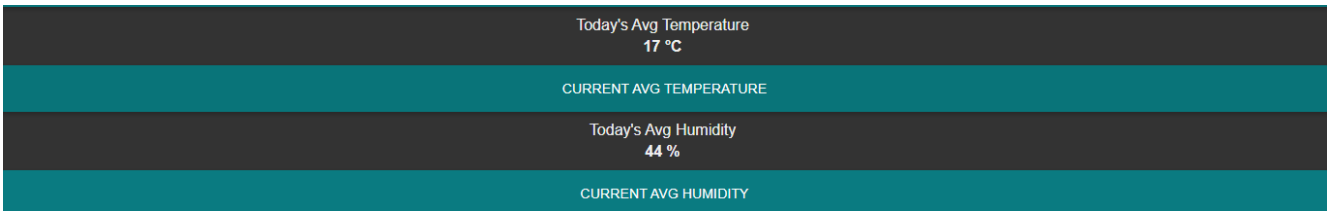


Fig 4.17. Buttons and Text Display of Average Temperature and Humidity

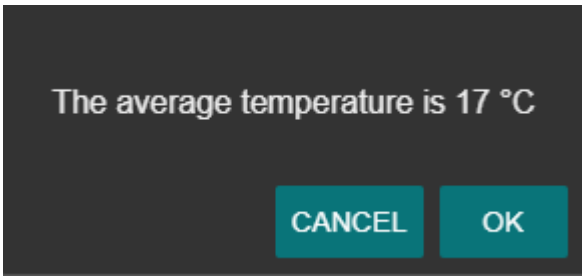


Fig 4.18. Pop-up window displaying average temperature

5. DATA STORAGE

Data is stored in InfluxDB within a specific bucket associated with a particular organization. A bucket (Fig 5.1.) is a designated storage location for time-series data, each with a defined retention period that determines how long data points are preserved. InfluxDB automatically deletes any data points with timestamps exceeding the bucket's retention period.

An organization (Fig. 5.2.) in InfluxDB serves as a workspace for a group of users, encompassing all resources such as dashboards, tasks, buckets, and members, which are managed within that organization.

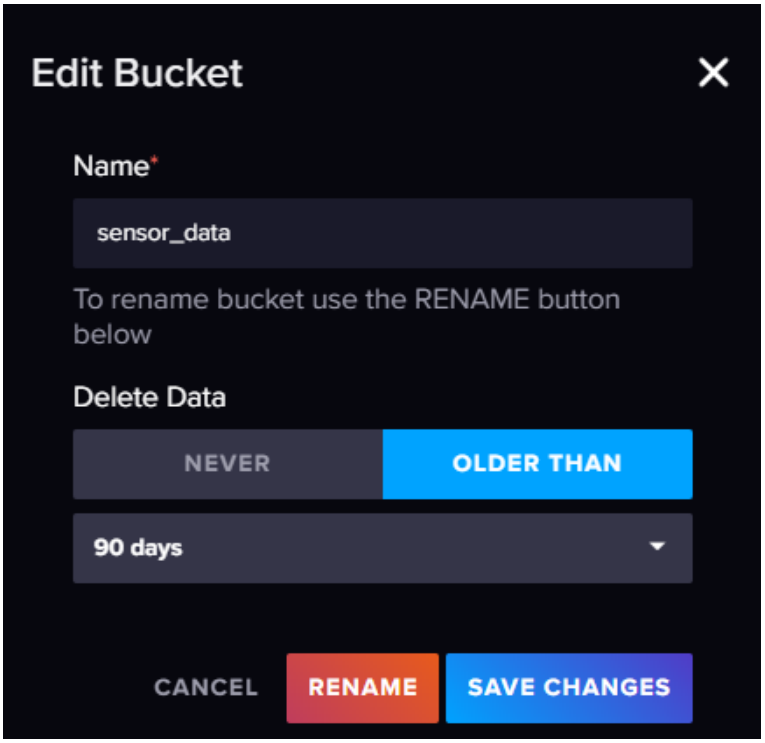


Fig. 5.1. Bucket Details

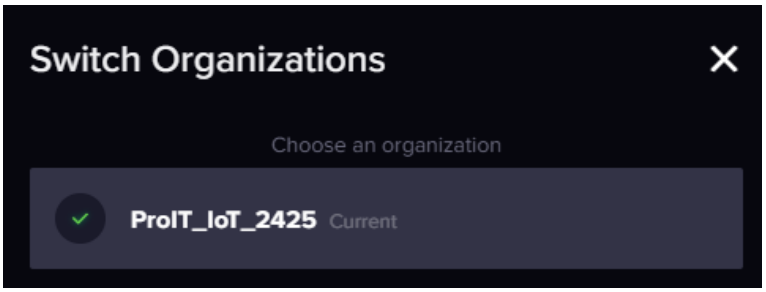


Fig. 5.2. Current Organization Details

Figs. 5.3 and 5.4 illustrate the configuration settings for the bucket and the InfluxDB endpoint. The bucket setup defines the storage location for time-series data, including measurement name and organizational context. The InfluxDB endpoint settings specify the connection parameters required for integrating the database with external tools or applications, enabling seamless data retrieval and visualization.

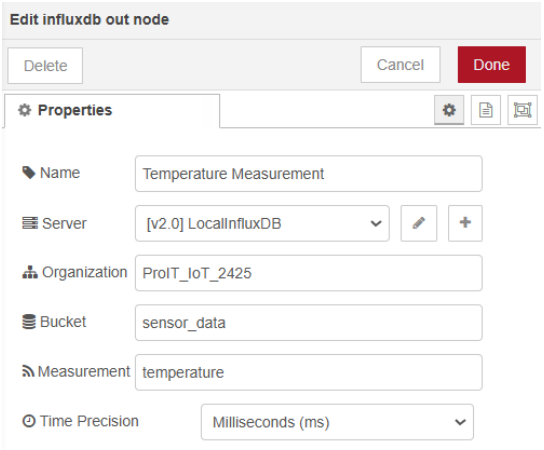


Fig. 5.3. Setting the server for Database

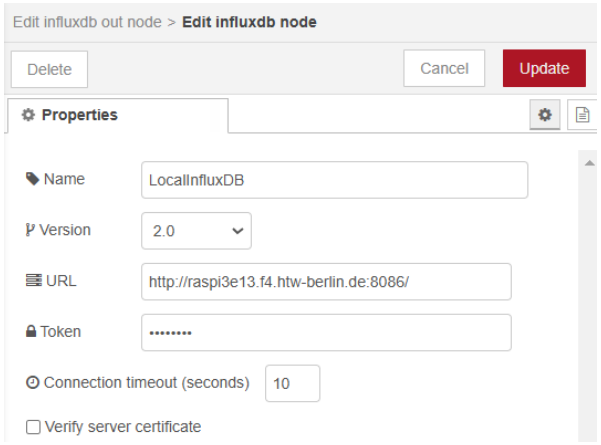


Fig. 5.4. Setting Properties of Database

5.1. Data Storage Flow

Fig. 5.3 displays how data flows from the MQTT IN node through the LINK OUT node to the LINK IN node as shown in the subsequent figure. The data is then formatted into a specific JSON object by including the temperature or humidity value with an appropriate tag in the payload. The payload is then transferred to InfluxDB with the use of INFLUXDB OUT node.

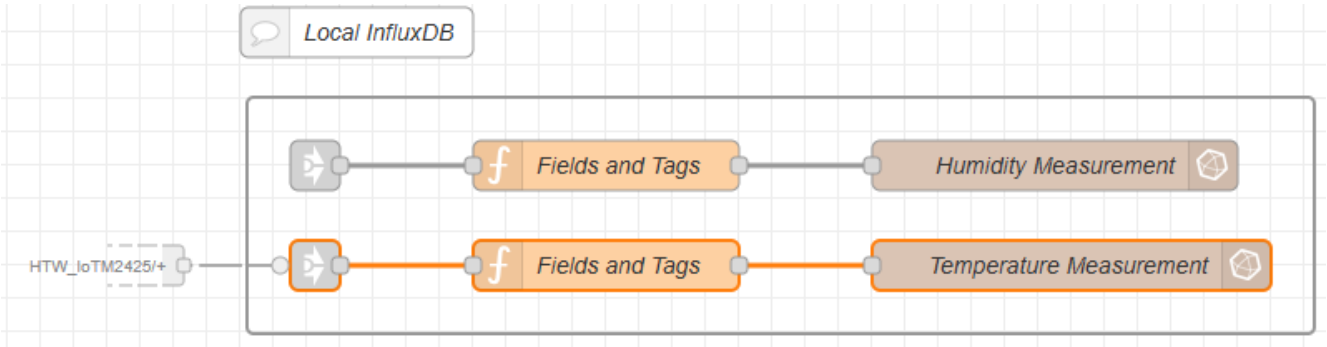


Fig. 5.5. Flow of data from MQTT broker to InfluxDB through the use of Link in and Link out node

6. UI AND VISUALIZATION

For this project, Node-red Dashboard and InfluxDB 2.0 was used for Visualization.

6.1. Data Visualization in Node-RED Dashboard

Figs. 6.1 and 6.2 showcase the Node-RED Dashboard, which directly receives inputs from the MQTT IN node (as referenced in Figs. 4.4 and 4.5). Fig. 6.1 presents data subscribed to using a wildcard topic, while Fig. 6.2 illustrates data subscribed to a specific topic.

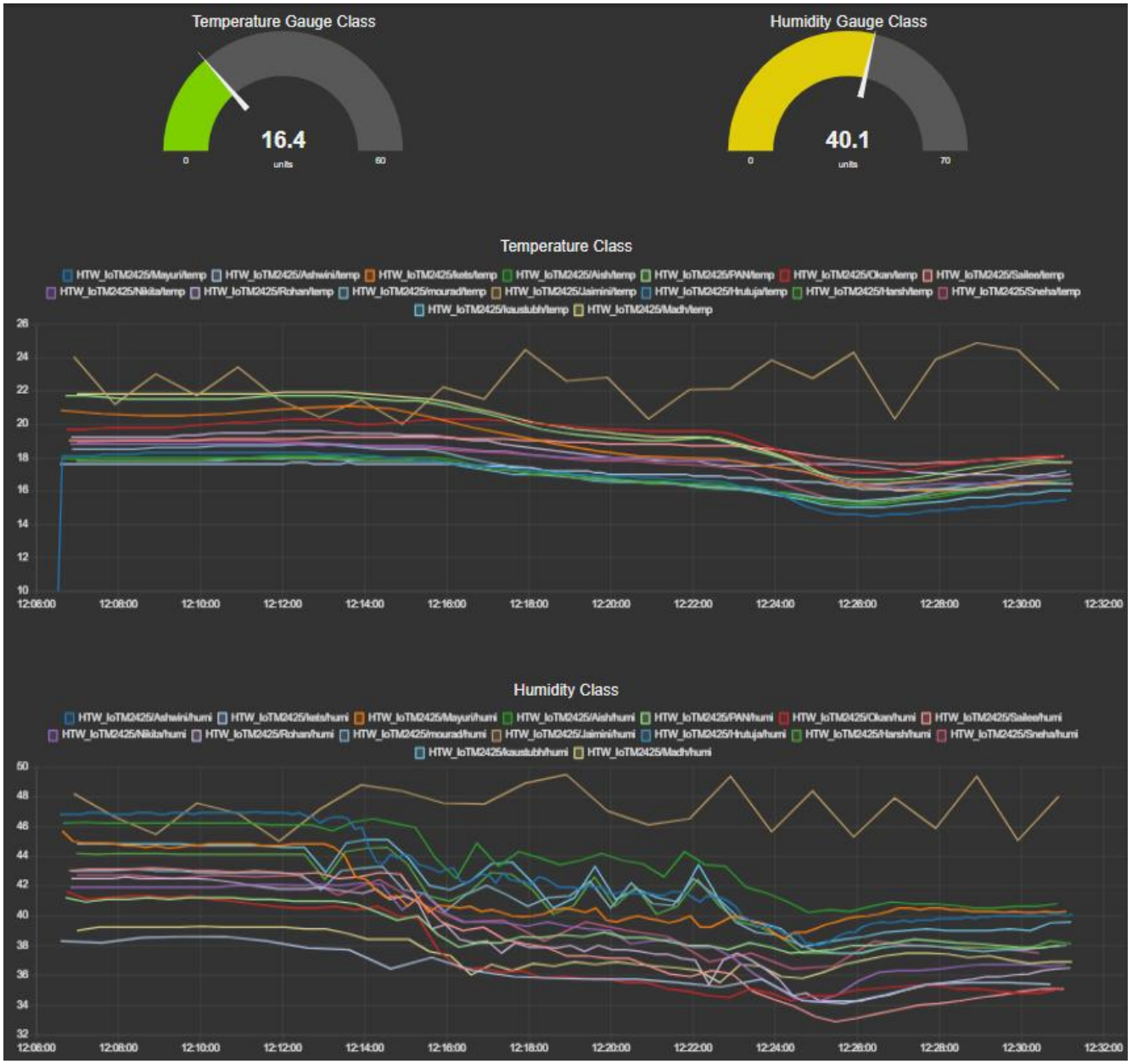


Fig. 6.1. Node Red Dashboard visualization subscribed to multiple topic

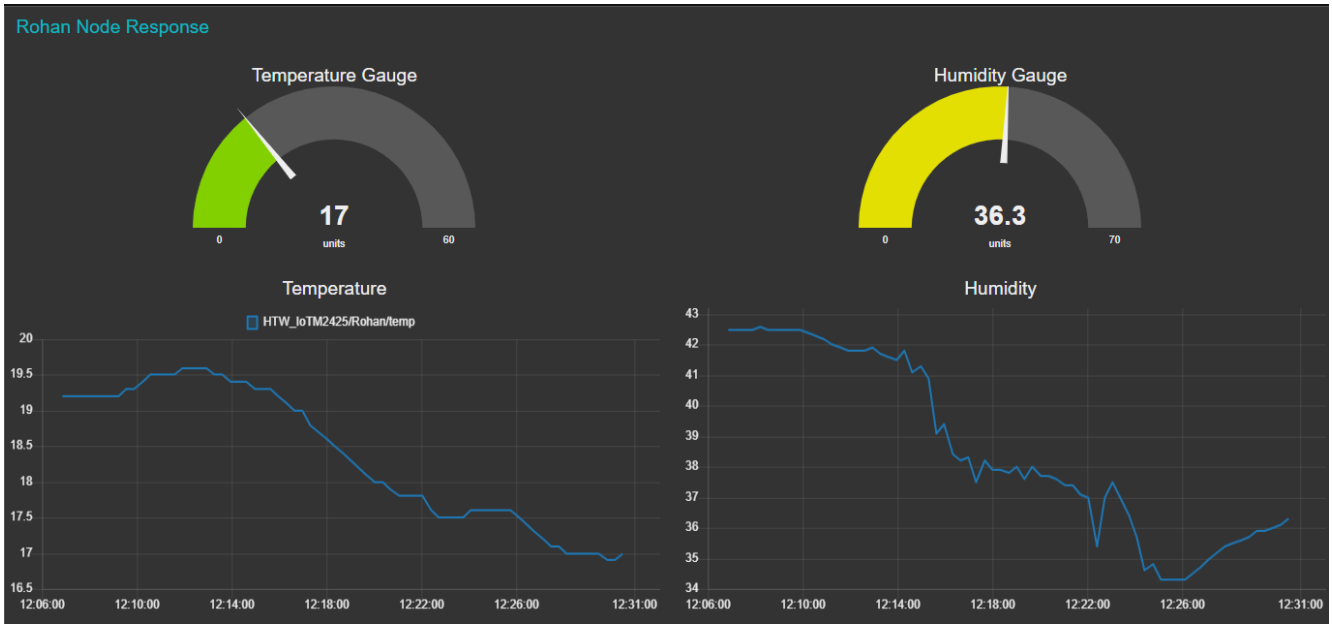


Fig. 6.2. Node Red Dashboard visualization subscribed to specific topic

6.2. Data Visualization in Influx DB:

InfluxDB is a time-series database designed for handling large volumes of time-series data, making it particularly well-suited for IoT applications, especially in this project, for data storage and visualization. Figs. 6.3. and 6.4. shows temperature and humidity line graph.

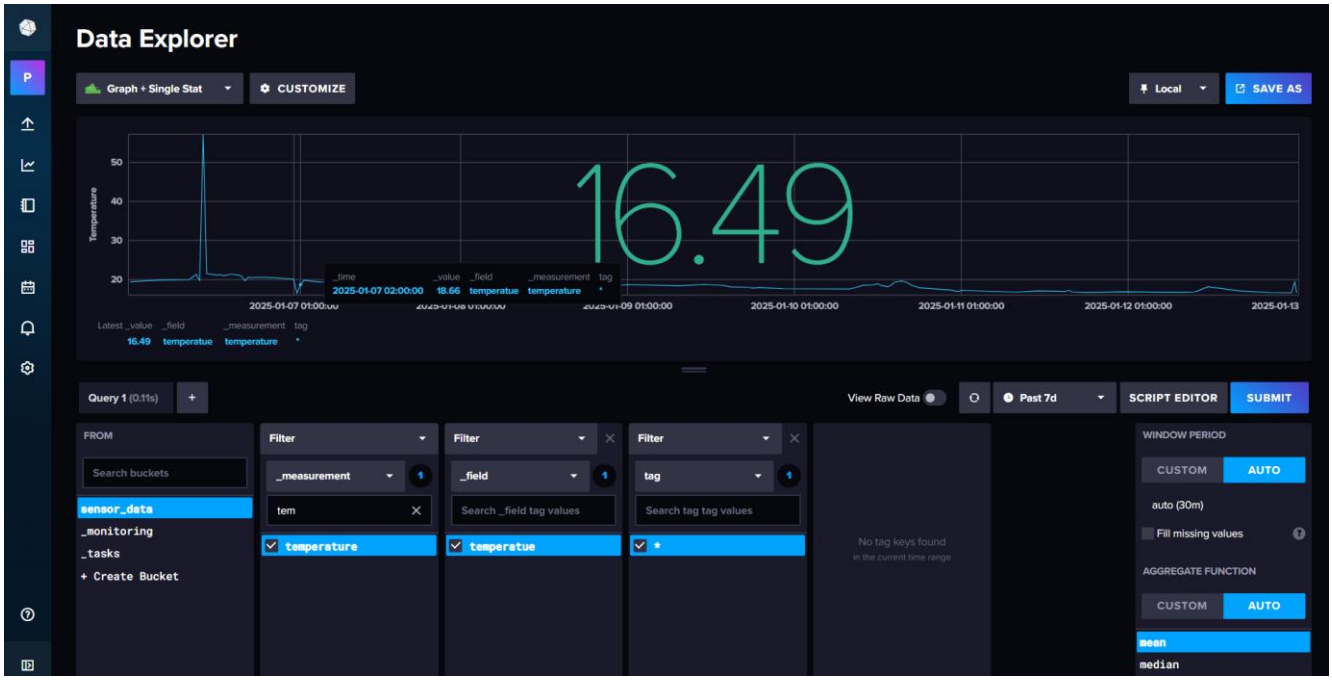


Fig. 6.3. Temperature values in stored data

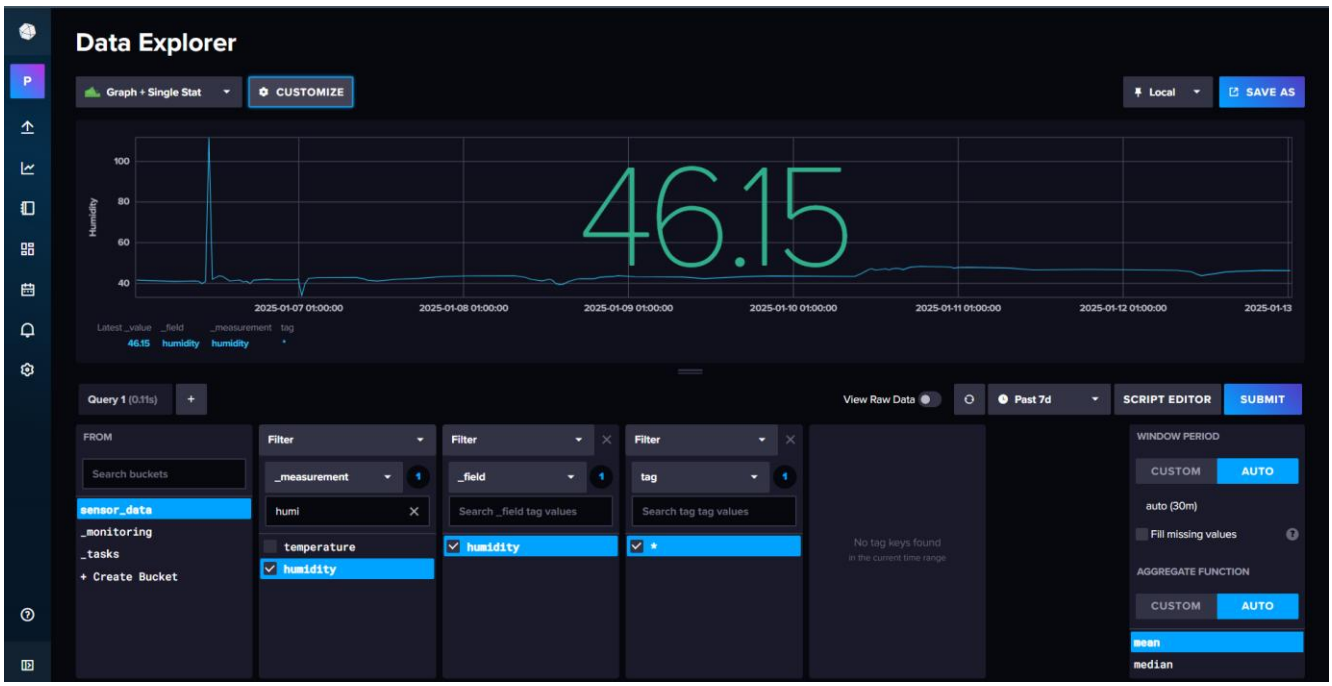


Fig. 6.4. Humidity values in stored data

7. Conclusion

The Telegram-based chatbot developed in this project serves as a convenient and efficient tool for monitoring temperature and humidity. It is user-friendly and can be seamlessly integrated into various settings, such as homes, greenhouses, and other environments where temperature and humidity are critical factors. The chatbot's functionality can be further enhanced by incorporating additional features and integrating it with other APIs for expanded capabilities.

8. References

1. MQTT: https://www.hivemq.com/blog/mqtt-essentials-part-1-introducing-mqtt/
2. InfluxDb Documentation: https://docs.influxdata.com/influxdb/v2/
3. Telegram Bot documentation: https://core.telegram.org/bots
4. Node-RED Documentation: https://nodered.org/docs/
5. Telegram Chatbot API Customization: https://github.com/hosein2398/node-telegram-bot-api-tutorial
6. NodeJS: https://www.w3schools.com/nodejs/

9. GitLab Repository

The project repository is hosted on gitlab. It contains all the source code, Node-RED flows, and documentation.

Gitlab Link: https://gitlab.rz.htw-berlin.de/s0595920/m_iot_ws2425_s0595920